

OSSP (Operating Systems And Systems Programming) - 25CS2104E

1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (uname, lscpu, lsblk, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

```
mokshitha_putti@MokshithaF x + - exp1.c *
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    char command[100];
    printf("Enter Linux Command: ");
    scanf("%s", command);

    pid_t pid = fork();
    if(pid == 0)
    {
        printf("Child PID: %d\n", getpid());
        execlp(command, command, NULL);
        perror("exec failed");
    }
    else if(pid > 0)
    {
        printf("Parent PID: %d\n", getpid());
        wait(NULL);
        printf("Child completed\n");
    }
    else
    {
        printf("Fork failed\n");
    }

    return 0;
}
```

```
mokshitha_putti@MokshithaF x + -
mokshitha_putti@MokshithaPutti:~$ nano exp1.c
mokshitha_putti@MokshithaPutti:~$ gcc exp1.c -o exp1
mokshitha_putti@MokshithaPutti:~$ ./exp1
Enter Linux Command: ls
Parent PID: 761
Child PID: 762
ANU          fork      hello      s2exampleprogram.c.save  week15
Demo         fork.c    hello.c    sample.txt               week15.c
OSLab        fork1     pid        shell                    week16
Week1        fork1.c  pid.c     week                      week16.c
Week1.c      fork2    q1         week12                   week17
Week12.c     fork2.c  q1.c      week12.c                 week17.c
copy.txt     fork3    q2         week13                   week18.c
example.txt  fork3.c  q2.c      week13.c                 week19
exp1         forkloop s2         week14                   week19.c
exp1.c       forkloop s2example.txt  week14.c
Child completed
mokshitha_putti@MokshithaPutti:~$
```

```
mokshitha_putti@Mokshithaf x + -
mokshitha_putti@MokshithaPutti:~$ nano expl.c
mokshitha_putti@MokshithaPutti:~$ gcc expl.c -o expl
mokshitha_putti@MokshithaPutti:~$ ./expl
Enter Linux Command: ls
Parent PID: 761
Child PID: 762
ANU      fork      hello      s2exampleprogram.c.save  week15
Demo     fork.c     hello.c     sample.txt               week15.c
OSLab    fork1      pid        shell                    week16
Week1    fork1.c    pid.c      week                      week16.c
Week1.c  fork2.c    q1         week12                   week17
Week12.c fork3.c    q2.c       week13.c                 week17.c
copy.txt  fork3.c    q2.c       week13.c                 week18.c
example.txt forkloop  s2         week14.c                 week19
expl.c   forkloop.c s2example.txt week14.c                 week19.c
Child completed
mokshitha_putti@MokshithaPutti:~$ uname
Linux
mokshitha_putti@MokshithaPutti:~$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          39 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 16
On-line CPU(s) list:   0-15
Vendor ID:              GenuineIntel
Model name:             13th Gen Intel(R) Core(TM) i7-13620H
CPU family:             6
Model:                 186
Thread(s) per core:     2
Core(s) per socket:     8
Socket(s):              1
Stepping:               2
BogoMIPS:               5836.80
Flags:                  fpu vme de pse tsc msr pae mce cx8 apic sep mtrr p
ge mca cmov pat pse36 clflush mmx fxsr sse sse2 ss
ht syscall nx pdpe1gb rdtscp lm constant_tsc rep_
good_nopl xtopology tsc_reliable nonstop_tsc cpuid
tsc_known_freq pni pclmulqdq vmx ssse3 fma cx16 p
cid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline
```

```
mokshitha_putti@Mokshithaf x + -
_timer aes xsave avx f16c rdrand hypervisor lahf_l
m abm 3dnowprefetch ssbd ibrs ibpb stibp ibrs_enha
nced tpr_shadow ept vpid ept_ad fsgsbase tsc_adjus
t bml1 avx2 smep bmi2 erms invpcid rdseed adx smap
clflushopt clwb sha_ni xsaveopt xsavec xgetbv1 xs
aves avx_vnni vnni umip waitpkg gfni vaes vpclmulq
dq rdpid movdiri movdir64b fsrm md_clear serialize
flush_lld arch_capabilities
Virtualization features:
Virtualization:         VT-x
Hypervisor vendor:      Microsoft
Virtualization type:    full
Caches (sum of all):
L1d:                    384 KiB (8 instances)
L1i:                    256 KiB (8 instances)
L2:                     10 MiB (8 instances)
L3:                     24 MiB (1 instance)
NUMA:
NUMA node(s):           1
NUMA node0 CPU(s):      0-15
Vulnerabilities:
Gather data sampling:    Not affected
Itlb multihit:           Not affected
L1tf:                    Not affected
Mds:                     Not affected
Meltdown:                Not affected
Mmio stale data:         Not affected
Reg file data sampling:  Mitigation; Clear Register File
Retbleed:                Mitigation; Enhanced IBRS
Spec rstack overflow:    Not affected
Spec store bypass:       Mitigation; Speculative Store Bypass disabled via
prctl
Spectre v1:              Mitigation; usercopy/swapgs barriers and __user po
inter sanitization
Spectre v2:              Mitigation; Enhanced / Automatic IBRS; IBPB condit
ional; RSB filling; PBSRB-eIBRS SW sequence; BHI B
HI_DIS_S
Srbds:                   Not affected
Tsx async abort:         Not affected
mokshitha_putti@MokshithaPutti:~$ lsblk
NAME MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
```

```
mokshitha_putti@Mokshithaf x + v - □ ×
top - 12:38:33 up 24 min, 1 user, load average: 0.00, 0.00, 0.00
Tasks: 25 total, 1 running, 24 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni, 100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.
MiB Mem : 7788.2 total, 7169.7 free, 566.0 used, 204.9 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used, 7222.2 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM    TIME+
    1 root        20   0   24272   14380  10924 S   0.0   0.2   0:01.23
    2 root        20   0    3072    1792   1792 S   0.0   0.0   0:00.01
    7 root        20   0    3072    1792   1792 S   0.0   0.0   0:00.00
   50 root        19  -1   50384  16000  14976 S   0.0   0.2   0:00.25
   84 systemd+    20   0   22400   14080  11648 S   0.0   0.2   0:00.27
   92 root        20   0   35248  11904   8960 S   0.0   0.1   0:01.07
  110 root        20   0    2880    1664   1664 S   0.0   0.0   0:00.05
  111 root        20   0    4464    2816   2688 S   0.0   0.0   0:00.00
  112 message+    20   0    8796    4608   4224 S   0.0   0.1   0:00.07
  118 root        20   0   44080  27892  13568 S   0.0   0.3   0:00.24
  124 root        20   0   18420   8576   7552 S   0.0   0.1   0:00.14
  157 root        20   0    5528    2688   2560 S   0.0   0.0   0:00.01
  188 syslog     20   0  220532  4992   4096 S   0.0   0.1   0:00.11
  225 root        20   0    5484    2560   2432 S   0.0   0.0   0:00.00
  231 _chrony      20   0   23716   9728   6016 S   0.0   0.1   0:00.12
  244 _chrony      20   0   12080    2232   1664 S   0.0   0.0   0:00.00
  352 root        20   0  123444  31744  17408 S   0.0   0.4   0:00.15
  402 root        20   0    3088     896     768 S   0.0   0.0   0:00.00
  403 root        20   0    3088    1160    1024 S   0.0   0.0   0:00.01
  409 mokshit+   20   0    6260    5248   3712 S   0.0   0.1   0:00.10
  412 root        20   0    8636    4864   4352 S   0.0   0.1   0:00.01
  458 mokshit+   20   0   22356  13056  10624 S   0.0   0.2   0:00.32
  460 mokshit+   20   0   22896    4032   2048 S   0.0   0.1   0:00.00
  486 mokshit+   20   0    6128    5248   3712 S   0.0   0.1   0:00.04
  820 mokshit+   20   0    8164    5760   3712 R   0.0   0.1   0:00.10
```

2. Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved.

```
mokshitha_putti@Mokshithaf x + v - □ ×
GNU nano 8.7.1 exp2.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int src, dest;
    char buffer[1024];
    int n;

    src = open("source.txt", O_RDONLY);
    dest = open("destination.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (src < 0 || dest < 0)
    {
        perror("File open error");
        return 1;
    }

    while ((n = read(src, buffer, sizeof(buffer))) > 0)
    {
        write(dest, buffer, n);
    }

    close(src);
    close(dest);

    printf("File copied successfully.\n");

    return 0;
}
```

```

mokshitha_putti@MokshithaPutti:~$ nano exp2.c
mokshitha_putti@MokshithaPutti:~$ echo "Operating Systems and Systems Progra
mming" > source.txt
mokshitha_putti@MokshithaPutti:~$ gcc exp2.c -o exp2
mokshitha_putti@MokshithaPutti:~$ ./exp2
File copied successfully.
mokshitha_putti@MokshithaPutti:~$ cat destination.txt
Operating Systems and Systems Programming
mokshitha_putti@MokshithaPutti:~$ strace cat source.txt
execve("/usr/bin/cat", ["cat", "source.txt"], 0x7fff419a7778 /* 25 vars */)
= 0
brk(NULL)
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0
x722679d0e000
access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directo
ry)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=20219, ...}) = 0
mmap(NULL, 20219, PROT_READ, MAP_PRIVATE, 3, 0) = 0x722679d09000
close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libselinux.so.1", O_RDONLY|O_CLO
EXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0"...
, 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=199120, ...}) = 0
mmap(NULL, 206288, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x722679cd6
000
mmap(0x722679cdd000, 135168, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_
DENYWRITE, 3, 0x7000) = 0x722679cdd000
mmap(0x722679cfe000, 28672, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x28000) = 0x722679cfe000
mmap(0x722679d05000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_D
ENYWRITE, 3, 0x2f000) = 0x722679d05000
mmap(0x722679d07000, 5584, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_A
NONYMOUS, -1, 0) = 0x722679d07000
close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libgcc_s.so.1", O_RDONLY|O_CLOEX
EC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0"...
, 832) = 832

```

```

mokshitha_putti@MokshithaPutti:~$ cat source.txt
statx(AT_FDCWD, "/usr/share/coreutils/locales/cat", AT_STATX_SYNC_AS_STAT, S
TATX_ALL, {stx_mask=STATX_ALL|STATX_MNT_ID, stx_attributes=0, stx_mode=S_IFD
IR|0755, stx_size=4096, ...}) = 0
openat(AT_FDCWD, "/usr/share/coreutils/locales/cat/en-US.ftl", O_RDONLY|O_CL
OEXEC) = -1 ENOENT (No such file or directory)
rt_sigaction(SIGPIPE, {sa_handler=SIG_DFL, sa_mask=[PIPE], sa_flags=SA_RESTO
RER|SA_RESTART, sa_restorer=0x722679845cb0}, {sa_handler=SIG_IGN, sa_mask=[P
IPE], sa_flags=SA_RESTORE|SA_RESTART, sa_restorer=0x722679845cb0}, 8) = 0
rt_sigaction(SIGSEGV, {sa_handler=SIG_DFL, sa_mask=[RTMIN RT_1], sa_flags=SA
_RESTORE|SA_RESTART, sa_restorer=0x722679845cb0}, {sa_handler=0x57167170b0c0, sa_mask=
[], sa_flags=SA_RESTORE|SA_ONSTACK|SA_SIGINFO, sa_restorer=0x722679845cb0},
8) = 0
rt_sigaction(SIGBUS, {sa_handler=SIG_DFL, sa_mask=[RTMIN RT_1], sa_flags=SA
_RESTORE|SA_RESTART, sa_restorer=0x722679845cb0}, {sa_handler=0x57167170b0c0, sa_mask=
[], sa_flags=SA_RESTORE|SA_ONSTACK|SA_SIGINFO, sa_restorer=0x722679845cb0},
8) = 0
ioctl(C1, TCGETS2, {c_iflag=TCRNL|IXON, c_oflag=NL0|CR0|TAB0|BS0|VT0|FF0|OPOS
T|ONLCR, c_cflag=B38400|C58|CREAD, c_lflag=ISIG|ICANON|ECHO|ECHOE|ECHO|IEXT
EN|ECHOC|ECHOK, ...}) = 0
brk(0x571682ef3000) = 0x571682ef3000
statx(AT_FDCWD, "source.txt", AT_STATX_SYNC_AS_STAT, STATX_ALL, {stx_mask=ST
ATX_ALL|STATX_MNT_ID, stx_attributes=0, stx_mode=S_IFREG|0644, stx_size=42,
...}) = 0
openat(AT_FDCWD, "source.txt", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=42, ...}) = 0
fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0), ...}) = 0
splice(3, NULL, 1, NULL, 1048576, 0) = -1 EINVAL (Invalid argument)
pipe([4, 5]) = 0
fcntl(4, F_SETPIPE_SZ, 1048576) = 1048576
splice(3, NULL, 1, NULL, 1048576, 0) = 42
splice(4, NULL, 1, NULL, 42, 0) = 42
Operating Systems and Systems Programming
splice(3, NULL, 5, NULL, 1048576, 0) = 0
close(5) = 0
close(4) = 0
close(3) = 0
sigaltstack({ss_sp=NULL, ss_flags=SS_DISABLE, ss_size=8192}, NULL) = 0
munmap(0x722679d0b000, 12288) = 0
exit_group(0) = ?
+++ exited with 0 +++
mokshitha_putti@MokshithaPutti:~$

```

3. Develop a C program using `fork()` that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as `ps`, `top`, and `/proc`. Document the observations.

```
GNU nano 8.7.1 exp3.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid = fork();
    if (pid == 0)
    {
        printf("Child Process\n");
        printf("PID: %d\n", getpid());
        printf("PPID: %d\n", getppid());

        sleep(5);

        printf("Child process completed\n");
    }
    else if (pid > 0)
    {
        printf("Parent Process\n");
        printf("PID: %d\n", getpid());
        printf("PPID: %d\n", getppid());

        wait(NULL);

        printf("Parent process completed\n");
    }
    else
    {
        perror("fork");
    }

    return 0;
}
```

```
mokshitha_putti@MokshithaPutti:~$ nano exp3.c
mokshitha_putti@MokshithaPutti:~$ gcc exp3.c -o exp3
mokshitha_putti@MokshithaPutti:~$ ./exp3
Parent Process
PID: 973
PPID: 409
Child Process
PID: 974
PPID: 973
Child process completed
Parent process completed
mokshitha_putti@MokshithaPutti:~$ ps -ef
UID          PID     PPID  C  STIME TTY          TIME CMD
root          1         0  0   12:14 ?        00:00:01 /sbin/init
root          2         1  0   12:14 ?        00:00:00 /init
root          7         2  0   12:14 ?        00:00:00 plan9 --control-socket 7
root         50         1  0   12:14 ?        00:00:00 /usr/lib/systemd/systemd
systemd+     84         1  0   12:14 ?        00:00:00 /usr/lib/systemd/systemd
root         92         1  0   12:14 ?        00:00:01 /usr/lib/systemd/systemd
root        110         1  0   12:14 ?        00:00:00 /bin/sh /usr/lib/systemd
root        111         1  0   12:14 ?        00:00:00 /usr/sbin/cron -f -p
message+    112         1  0   12:14 ?        00:00:00 @dbus-daemon --system --
root        118         1  0   12:14 ?        00:00:00 /usr/bin/python3 /usr/bi
root        124         1  0   12:14 ?        00:00:00 /usr/lib/systemd/systemd
root        157         1  0   12:14 hvcs0    00:00:00 /usr/sbin/agetty --nores
syslog      188         1  0   12:14 ?        00:00:00 /usr/sbin/rsyslogd -n -i
root        225         1  0   12:14 tty1     00:00:00 /usr/sbin/agetty --nores
_chrony     231        110  0   12:14 ?        00:00:00 /usr/sbin/chronyd -n -F
_chrony     244        231  0   12:14 ?        00:00:00 /usr/sbin/chronyd -n -F
root        352         1  0   12:14 ?        00:00:00 /usr/bin/python3 /usr/sh
root        402         2  0   12:14 ?        00:00:00 /init
root        403         2  0   12:14 ?        00:00:00 /init
mokshit+   409        403  0   12:14 pts/0    00:00:00 -bash
root        412         2  0   12:14 ?        00:00:00 login -- mokshitha_putti
mokshit+   458         1  0   12:14 ?        00:00:00 /usr/lib/systemd/systemd
mokshit+   460        458  0   12:14 ?        00:00:00 (sd-pam)
mokshit+   486        412  0   12:14 pts/1    00:00:00 -bash
root        913         1  0   12:45 ?        00:00:00 /usr/libexec/wsl-pro-ser
mokshit+   994        409  0   12:47 pts/0    00:00:00 ps -ef
mokshitha_putti@MokshithaPutti:~$ top
top - 12:47:24 up 32 min, 1 user, load average: 0.05, 0.01, 0.00
```

```
mokshitha_putti@MokshithaPutti:~$ cat /proc/<PID>/status
-bash: PID: No such file or directory
mokshitha_putti@MokshithaPutti:~$
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+
1	root	20	0	24272	14380	10924	S	0.0	0.2	0:01.26
2	root	20	0	3072	1792	1792	S	0.0	0.0	0:00.01
7	root	20	0	3072	1792	1792	S	0.0	0.0	0:00.00
50	root	19	-1	50384	16000	14976	S	0.0	0.2	0:00.26
84	systemd+	20	0	22400	14080	11648	S	0.0	0.2	0:00.27
92	root	20	0	35248	11904	8960	S	0.0	0.1	0:01.23
110	root	20	0	2880	1664	1664	S	0.0	0.0	0:00.05
111	root	20	0	4464	2816	2688	S	0.0	0.0	0:00.01
112	message+	20	0	8796	4608	4224	S	0.0	0.1	0:00.07
118	root	20	0	44080	27892	13568	S	0.0	0.3	0:00.24
124	root	20	0	18420	8576	7552	S	0.0	0.1	0:00.14
157	root	20	0	5528	2688	2560	S	0.0	0.0	0:00.01
188	syslog	20	0	220532	4992	4096	S	0.0	0.1	0:00.11
225	root	20	0	5484	2560	2432	S	0.0	0.0	0:00.00
231	_chrony	20	0	23716	9728	6016	S	0.0	0.1	0:00.16
244	_chrony	20	0	12080	2232	1664	S	0.0	0.0	0:00.00
352	root	20	0	123444	31744	17408	S	0.0	0.4	0:00.15
402	root	20	0	3088	896	768	S	0.0	0.0	0:00.00
403	root	20	0	3088	1160	1024	S	0.0	0.0	0:00.05
409	mokshit+	20	0	6260	5248	3712	S	0.0	0.1	0:00.15
412	root	20	0	8636	4864	4352	S	0.0	0.1	0:00.01
458	mokshit+	20	0	22356	13056	10624	S	0.0	0.2	0:00.32
460	mokshit+	20	0	22896	4032	2048	S	0.0	0.1	0:00.00
486	mokshit+	20	0	6128	5248	3712	S	0.0	0.1	0:00.04
913	root	20	0	179222	14080	11520	S	0.0	0.2	0:00.08
994	mokshit+	20	0	8164	5760	3712	R	0.0	0.1	0:00.00

4. Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
mokshitha_putti@Mokshithaf x + v
GNU nano 8.7.1 exp4.c *
{
    printf("Child 3: PID = %d\n", getpid());
    sleep(6);
    return 30;
}

printf("Parent PID = %d\n", getpid());
printf("Using waitpid() for Child 1\n");
waitpid(p1, NULL, 0);

printf("Using wait() for remaining children\n");
wait(NULL);
wait(NULL);

printf("All children completed.\n");
return 0;
}|
```

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();
    if (pid == 0)
    {
        printf("Child exiting. PID = %d\n", getpid());
        return 0;
    }
    else
    {
        printf("Parent PID = %d\n", getpid());
        sleep(20);
    }
    return 0;
}|
```

```

mokshitha_putti@MokshithaF x + -
mokshitha_putti@MokshithaPutti:~$ cat /proc/<PID>/status
-bash: PID: No such file or directory
mokshitha_putti@MokshithaPutti:~$ nano exp4.c
mokshitha_putti@MokshithaPutti:~$ gcc exp4.c -o exp4
mokshitha_putti@MokshithaPutti:~$ ./exp4
Child 1: PID = 1050
Child 2: PID = 1051
Parent PID = 1049
Using waitpid() for Child 1
Child 3: PID = 1052
Using wait() for remaining children
All children completed.
mokshitha_putti@MokshithaPutti:~$ nano zombie.c
mokshitha_putti@MokshithaPutti:~$ gcc zombie.c -o zombie
mokshitha_putti@MokshithaPutti:~$ ./zombie
Parent PID = 1122
Child exiting. PID = 1123
ps -ef
^Z
[1]+  Stopped                  ./zombie
mokshitha_putti@MokshithaPutti:~$ ps -ef

```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	1	0	0	12:41	?	00:00:01	/sbin/init
root	2	1	0	12:41	?	00:00:00	/init
root	7	2	0	12:41	?	00:00:00	plan9 --control-socket 7
root	50	1	0	12:41	?	00:00:00	/usr/lib/systemd/systemd
systemd+	84	1	0	12:41	?	00:00:00	/usr/lib/systemd/systemd
root	92	1	0	12:41	?	00:00:01	/usr/lib/systemd/systemd
root	110	1	0	12:41	?	00:00:00	/bin/sh /usr/lib/systemd
root	111	1	0	12:41	?	00:00:00	/usr/sbin/cron -f -P
message+	112	1	0	12:41	?	00:00:00	@dbus-daemon --system --
root	118	1	0	12:41	?	00:00:00	/usr/bin/python3 /usr/bi
root	124	1	0	12:41	?	00:00:00	/usr/lib/systemd/systemd
root	157	1	0	12:41	hvc0	00:00:00	/usr/sbin/agetty --nores
syslog	188	1	0	12:41	?	00:00:00	/usr/sbin/rsyslogd -n -i
root	225	1	0	12:41	tty1	00:00:00	/usr/sbin/agetty --nores
_chrony	231	110	0	12:41	?	00:00:00	/usr/sbin/chronyd -n -F
_chrony	244	231	0	12:41	?	00:00:00	/usr/sbin/chronyd -n -F
root	352	1	0	12:41	?	00:00:00	/usr/bin/python3 /usr/sh
root	402	2	0	12:41	?	00:00:00	/init
root	403	402	0	12:41	?	00:00:00	/init

5. Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

```

GNU nano 8.7.1 exp5.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    char buffer[100];

    pipe(pipefd);

    pid_t pid = fork();

    if (pid == 0)
    {
        close(pipefd[1]);

        read(pipefd[0], buffer, sizeof(buffer));
        printf("Child consumed: %s\n", buffer);

        close(pipefd[0]);
    }
    else
    {
        close(pipefd[0]);

        char message[] = "Data produced by parent";
        write(pipefd[1], message, sizeof(message));

        close(pipefd[1]);
        wait(NULL);

        printf("Parent produced data successfully.\n");
    }

    return 0;
}

```

^G Help ^O Write Out ^L Read 37 lines ^K Cut ^T Execute
 ^X Exit ^R Read File ^_ Where Is ^U Paste ^J Justify

```

GNU nano 8.7.1                                     pipegrep.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int fd[2];
    pipe(fd);

    pid_t p1 = fork();

    if (p1 == 0)
    {
        dup2(fd[1], STDOUT_FILENO);
        close(fd[0]);
        close(fd[1]);

        execlp("ls", "ls", "-l", NULL);
        return 1;
    }

    pid_t p2 = fork();

    if (p2 == 0)
    {
        dup2(fd[0], STDIN_FILENO);
        close(fd[1]);
        close(fd[0]);

        execlp("grep", "grep", ".c", NULL);
        return 1;
    }

    close(fd[0]);
    close(fd[1]);

    wait(NULL);
}

```

```

mokshitha_putti@MokshithaPutti:~$ nano exp5.c
mokshitha_putti@MokshithaPutti:~$ gcc exp5.c -o exp5
mokshitha_putti@MokshithaPutti:~$ ./exp5
Child consumed: Data produced by parent
Parent produced data successfully.
mokshitha_putti@MokshithaPutti:~$ nano pipegrep.c
mokshitha_putti@MokshithaPutti:~$ gcc pipegrep.c -o pipegrep
mokshitha_putti@MokshithaPutti:~$ ./pipegrep
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 77 Aug 4 08:45 Week1.c
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 158 Jul 31 09:57 Week12.c
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 47 Aug 8 16:34 copy.txt
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 560 Aug 15 12:32 exp1.c
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 529 Aug 15 12:42 exp2.c
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 590 Aug 15 12:46 exp3.c
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 760 Aug 15 12:51 exp4.c
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 615 Aug 15 13:28 exp5.c
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 336 Aug 14 09:51 fork.c
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 418 Aug 14 10:00 fork1.c
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 376 Aug 14 10:05 fork2.c
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 288 Aug 14 10:09 fork3.c
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 246 Aug 14 10:15 forkloop.c
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 80 Aug 4 03:49 hello.c
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 182 Aug 14 09:38 pid.c
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 580 Aug 15 13:29 pipegrep.c
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 738 Aug 8 15:35 q1.c
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 740 Aug 8 16:33 q2.c
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 77 Jul 31 09:16 s2examplepro
gram.c.save
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 42 Aug 15 12:42 source.txt
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 158 Aug 4 08:57 week12.c
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 110 Aug 4 08:59 week13.c
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 104 Aug 4 09:01 week14.c
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 230 Aug 4 09:02 week15.c
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 287 Aug 4 09:03 week16.c
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 274 Aug 4 09:04 week17.c
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 161 Aug 4 09:08 week18.c
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 264 Aug 4 09:10 week19.c
-rw-r--r-- 1 mokshitha_putti mokshitha_putti 285 Aug 15 13:24 zombie.c
mokshitha_putti@MokshithaPutti:~$

```

6. Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

- Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

Make use of the /proc//maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

```

GNU nano 8.7.1 exp7.c *
#include <stdio.h>
#include <stdlib.h>

int global_var = 10;
static int static_var = 20;

void function()
{
    int code_var = 30;
    printf("Code address : %p\n", (void *)function);
    printf("Stack address : %p\n", (void *)&code_var);
}

int main()
{
    int stack_var = 40;
    int *heap_var = malloc(sizeof(int));

    *heap_var = 50;

    printf("Code address : %p\n", (void *)main);
    printf("Global address : %p\n", (void *)&global_var);
    printf("Static address : %p\n", (void *)&static_var);
    printf("Heap address : %p\n", (void *)&heap_var);
    printf("Stack address : %p\n", (void *)&stack_var);

    function();

    free(heap_var);

    return 0;
}

```

```

mokshitha_putti@MokshithaPutti:~$ nano exp7.c
mokshitha_putti@MokshithaPutti:~$ gcc exp7.c -o exp7
mokshitha_putti@MokshithaPutti:~$ ./exp7
Code address : 0x61ee7e30721b
Global address : 0x61ee7e30a010
Static address : 0x61ee7e30a014
Heap address : 0x61eea4a2f010
Stack address : 0x7fff2bb8944c
Code address : 0x61ee7e3071a9
Stack address : 0x7fff2bb89424
mokshitha_putti@MokshithaPutti:~$ sleep 30 &
[1] 455
mokshitha_putti@MokshithaPutti:~$ ps
PID TTY          TIME CMD
328 pts/0        00:00:00 bash
455 pts/0        00:00:00 sleep
465 pts/0        00:00:00 ps
mokshitha_putti@MokshithaPutti:~$ cat /proc/455/maps
cat: /proc/455/maps: No such file or directory
[1]+  Done                  sleep 30
mokshitha_putti@MokshithaPutti:~$ sleep 300 &
[1] 485
mokshitha_putti@MokshithaPutti:~$ ps
PID TTY          TIME CMD
328 pts/0        00:00:00 bash
485 pts/0        00:00:00 sleep
495 pts/0        00:00:00 ps
mokshitha_putti@MokshithaPutti:~$ cat /proc/485/maps
5cb51b421000-5cb51b513000 r--p 00000000 08:30 1584 /us
r/lib/cargo/bin/coreutils/sleep
5cb51b513000-5cb51b9cb000 r-xp 000f2000 08:30 1584 /us
r/lib/cargo/bin/coreutils/sleep
5cb51b9cb000-5cb51bd8a000 r--p 005aa000 08:30 1584 /us
r/lib/cargo/bin/coreutils/sleep
5cb51bd8a000-5cb51beef000 r--p 00968000 08:30 1584 /us
r/lib/cargo/bin/coreutils/sleep
5cb51beef000-5cb51bef5000 rw-p 00acd000 08:30 1584 /us
r/lib/cargo/bin/coreutils/sleep
5cb51bef5000-5cb51bef6000 rw-p 00000000 00:00 0 [he
5cb548e39000-5cb548e5a000 rw-p 00000000 00:00 0

```

```

741710ae3000-741710ae7000 r--p 000000000 08:30 14274
r/lib/x86_64-linux-gnu/libgcc_s.so.1
741710ae7000-741710b0b000 r-xp 000040000 08:30 14274
r/lib/x86_64-linux-gnu/libgcc_s.so.1
741710b0b000-741710b0f000 r--p 000280000 08:30 14274
r/lib/x86_64-linux-gnu/libgcc_s.so.1
741710b0f000-741710b10000 r--p 0002c0000 08:30 14274
r/lib/x86_64-linux-gnu/libgcc_s.so.1
741710b10000-741710b11000 rw-p 0002d0000 08:30 14274
r/lib/x86_64-linux-gnu/libgcc_s.so.1
741710b11000-741710b18000 r--p 000000000 08:30 14477
r/lib/x86_64-linux-gnu/libselinux.so.1
741710b18000-741710b39000 r-xp 000070000 08:30 14477
r/lib/x86_64-linux-gnu/libselinux.so.1
741710b39000-741710b40000 r--p 000280000 08:30 14477
r/lib/x86_64-linux-gnu/libselinux.so.1
741710b40000-741710b41000 r--p 0002f0000 08:30 14477
r/lib/x86_64-linux-gnu/libselinux.so.1
741710b41000-741710b42000 rw-p 000300000 08:30 14477
r/lib/x86_64-linux-gnu/libselinux.so.1
741710b42000-741710b44000 rw-p 000000000 00:00 0
741710b46000-741710b47000 --p 000000000 00:00 0
741710b47000-741710b4b000 rw-p 000000000 00:00 0
741710b4b000-741710b4c000 r--p 000000000 08:30 13638
r/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
741710b4c000-741710b7b000 r-xp 000010000 08:30 13638
r/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
741710b7b000-741710b86000 r--p 000300000 08:30 13638
r/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
741710b86000-741710b88000 r--p 0003b0000 08:30 13638
r/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
741710b88000-741710b89000 rw-p 0003d0000 08:30 13638
r/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
741710b89000-741710b8a000 rw-p 000000000 00:00 0
7fff294cf000-7fff294f0000 rw-p 000000000 00:00 0
ack]
7fff294f3000-7fff294f7000 r--p 000000000 00:00 0
ar]
7fff294f7000-7fff294f9000 r-xp 000000000 00:00 0
so]
mokshitha_putti@MokshithaPutti:~$ |

```

- Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

```

GNU nano 8.7.1 exp8.c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *p, *q;

    p = (int *)malloc(5 * sizeof(int));
    q = (int *)calloc(5, sizeof(int));
    p = (int *)realloc(p, 10 * sizeof(int));

    free(p);
    free(q);

    printf("Memory allocation completed successfully.\n");

    return 0;
}

```

```

mokshitha_putti@MokshithaPutti:~$ nano exp8.c
mokshitha_putti@MokshithaPutti:~$ gcc exp8.c -o exp8
mokshitha_putti@MokshithaPutti:~$ nano exp8.c
mokshitha_putti@MokshithaPutti:~$ gcc exp8.c -o exp8
mokshitha_putti@MokshithaPutti:~$ ./exp8
Memory allocation completed successfully.
mokshitha_putti@MokshithaPutti:~$ valgrind --leak-check=full ./exp8
Command 'valgrind' not found, but can be installed with:
sudo snap install valgrind # version 3.26.0, or
sudo apt install valgrind # version 1:3.26.0-0ubuntu1
See 'snap info valgrind' for additional versions.
mokshitha_putti@MokshithaPutti:~$ sudo apt install valgrind
[sudo: authenticate] Password:
Installing:
valgrind

Installing dependencies:
gdb libdebuginfod-common libipt2
libbabeltrace1 libdebuginfod1t64 libsource-highlight-common
libc6-dbg libdw1t64 libsource-highlight4t64

Suggested packages:
gdb-doc gdbserver valgrind-mpi kcachegrind

Summary:
Upgrading: 0, Installing: 10, Removing: 0, Not Upgrading: 39
Download size: 27.1 MB
Space needed: 111 MB / 1024 GB available

Continue? [Y/n] y
Get:1 http://archive.ubuntu.com/ubuntu resolute/main amd64 libdebuginfod-com
mon all 0.194-4 [12.3 kB]
Get:2 http://archive.ubuntu.com/ubuntu resolute/main amd64 libdw1t64 amd64 0
.194-4 [285 kB]
Get:3 http://archive.ubuntu.com/ubuntu resolute/main amd64 libbabeltrace1 am
d64 1.5.11-5build1 [165 kB]
Get:4 http://archive.ubuntu.com/ubuntu resolute/main amd64 libdebuginfod1t64
amd64 0.194-4 [20.7 kB]
Get:5 http://archive.ubuntu.com/ubuntu resolute/main amd64 libipt2 amd64 2.1
.2-3 [49.2 kB]

Unpacking libc6-dbg:amd64 (2.43-2ubuntu2.3) ...
Selecting previously unselected package valgrind.
Preparing to unpack .../9-valgrind_1%3a3.26.0-0ubuntu1_amd64.deb ...
Unpacking valgrind (1:3.26.0-0ubuntu1) ...
Setting up libdebuginfod-common (0.194-4) ...
Setting up libc6-dbg:amd64 (2.43-2ubuntu2.3) ...
Setting up libdw1t64:amd64 (0.194-4) ...
Setting up libipt2 (2.1.2-3) ...
Setting up libbabeltrace1:amd64 (1.5.11-5build1) ...
Setting up valgrind (1:3.26.0-0ubuntu1) ...
Setting up libdebuginfod1t64:amd64 (0.194-4) ...
Setting up libsource-highlight4t64:amd64 (3.1.9-4.3build2) ...
Setting up gdb (17.1-2ubuntu1) ...
Processing triggers for man-db (2.13.1-1build1) ...
Processing triggers for libc-bin (2.43-2ubuntu2.3) ...
mokshitha_putti@MokshithaPutti:~$ valgrind --leak-check=full ./exp8
==831== Memcheck, a memory error detector
==831== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==831== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==831== Command: ./exp8
==831==
Memory allocation completed successfully.
==831==
==831== HEAP SUMMARY:
==831==   in use at exit: 0 bytes in 0 blocks
==831==   total heap usage: 4 allocs, 4 frees, 1,104 bytes allocated
==831==
==831== All heap blocks were freed -- no leaks are possible
==831==
==831== For lists of detected and suppressed errors, rerun with: -s
==831== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
[1]+  Done                  sleep 300
mokshitha_putti@MokshithaPutti:~$ nano cow.c
mokshitha_putti@MokshithaPutti:~$ gcc cow.c -o cow
mokshitha_putti@MokshithaPutti:~$ ./cow
Before fork: data[0] = 10
Child before modification: data[0] = 10
Child after modification: data[0] = 20
Parent after child modification: data[0] = 10
mokshitha_putti@MokshithaPutti:~$

```

9. Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

Develop a program that redirects standard input and output using file descriptors and dup2(). Demonstrate how shells implement I/O redirection.

```

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int src, dest;
    char buffer[1024];
    int n;

    src = open("source.txt", O_RDONLY);
    dest = open("copy.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (src < 0 || dest < 0)
    {
        perror("open");
        return 1;
    }

    while ((n = read(src, buffer, sizeof(buffer))) > 0)
    {
        write(dest, buffer, n);
    }

    lseek(src, 0, SEEK_SET);
    close(src);
    close(dest);

    printf("File copied successfully.\n");

    return 0;
}

```

```

mokshitha_putti@MokshithaPutti:~$ nano exp9.c
mokshitha_putti@MokshithaPutti:~$ nano exp9.c
mokshitha_putti@MokshithaPutti:~$ gcc exp9.c -o exp9
mokshitha_putti@MokshithaPutti:~$ ./exp9
File copied successfully.
mokshitha_putti@MokshithaPutti:~$ cat copy.txt
Operating Systems and Systems Programming
mokshitha_putti@MokshithaPutti:~$ nano redirect.c
mokshitha_putti@MokshithaPutti:~$ gcc redirect.c -o redirect
mokshitha_putti@MokshithaPutti:~$ ./redirect
mokshitha_putti@MokshithaPutti:~$ cat output.txt
This output is redirected to a file.
mokshitha_putti@MokshithaPutti:~$

```

10. Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity.

```

GNU nano 8.7.1 mmap.c *
#include <stdio.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <unistd.h>

int main()
{
    int fd = open("original.txt", O_RDONLY);

    struct stat st;
    fstat(fd, &st);

    char *data = mmap(NULL, st.st_size, PROT_READ, MAP_PRIVATE, fd, 0);
    write(STDOUT_FILENO, data, st.st_size);

    munmap(data, st.st_size);
    close(fd);

    return 0;
}

```

```

mokshitha_putti@MokshithaPutti:~$ echo "Hello Linux" > original.txt
mokshitha_putti@MokshithaPutti:~$ ls -i original.txt
16574 original.txt
mokshitha_putti@MokshithaPutti:~$ ln original.txt hardlink.txt
mokshitha_putti@MokshithaPutti:~$ ln -s original.txt symlink.txt
mokshitha_putti@MokshithaPutti:~$ ls -li original.txt hardlink.txt symlink.t
xt
16574 -rw-r--r-- 2 mokshitha_putti mokshitha_putti 12 Aug 16 12:29 hardlink.
txt
16574 -rw-r--r-- 2 mokshitha_putti mokshitha_putti 12 Aug 16 12:29 original.
txt
18013 lrwxrwxrwx 1 mokshitha_putti mokshitha_putti 12 Aug 16 12:29 symlink.t
xt -> original.txt
mokshitha_putti@MokshithaPutti:~$ stat original.txt
  File: original.txt
  Size: 12          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 16574         Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 1000/mokshitha_putti)   Gid: ( 1000/mokshi
tha_putti)
Access: 2026-08-16 12:29:02.541285052 +0000
Modify: 2026-08-16 12:29:02.541285052 +0000
Change: 2026-08-16 12:29:19.781283785 +0000
 Birth: 2026-08-16 12:29:02.541285052 +0000
mokshitha_putti@MokshithaPutti:~$ find . -inum $(ls -i original.txt | awk '{
print $1}')
./hardlink.txt
./original.txt
mokshitha_putti@MokshithaPutti:~$ nano mmap.c
mokshitha_putti@MokshithaPutti:~$ gcc mmap.c -o mmap
mokshitha_putti@MokshithaPutti:~$ ./mmap
Hello Linux
mokshitha_putti@MokshithaPutti:~$ |

```

11. Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

Modify the previous program using mutex locks (pthread_mutex_lock, pthread_mutex_unlock) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

```

GNU nano 8.7.1                               exp11.c *
#include <stdio.h>
#include <pthread.h>

int counter = 0;

void *increment(void *arg)
{
    for (int i = 0; i < 100000; i++)
        counter++;

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final counter = %d\n", counter);

    return 0;
}

```

```

GNU nano 8.7.1 exp11_mutex.c *
#include <stdio.h>
#include <pthread.h>

int counter = 0;
pthread_mutex_t lock;

void *increment(void *arg)
{
    for (int i = 0; i < 100000; i++)
    {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main()
{
    pthread_t t1, t2;
    pthread_mutex_init(&lock, NULL);
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("Final counter = %d\n", counter);
    pthread_mutex_destroy(&lock);
    return 0;
}

mokshitha_putti@MokshithaPutti:~$ nano exp11.c
mokshitha_putti@MokshithaPutti:~$ gcc exp11.c -o exp11 -pthread
mokshitha_putti@MokshithaPutti:~$ ./exp11
Final counter = 111564
mokshitha_putti@MokshithaPutti:~$ nano exp11_mutex.c
mokshitha_putti@MokshithaPutti:~$ gcc exp11_mutex.c -o exp11_mutex -pthread
mokshitha_putti@MokshithaPutti:~$ ./exp11_mutex
Final counter = 200000
mokshitha_putti@MokshithaPutti:~$ |

```

12. Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

```
GNU nano 8.7.1 exp12.c *
{
    pthread_t producer_thread, consumer_thread;

    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}|
```

```
GNU nano 8.7.1 deadlock.c *
    return NULL;
}
int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&lock1, NULL);
    pthread_mutex_init(&lock2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}|
```